Project Final Report

1. Cover Information

Project title: GarmentIQ
Team name: The Juls
Team Leader: Garv Dudy
Date: Friday, April 10th, 2026

2. Abstract

GarmentIQ is a web-based apparel operations and sales intelligence system designed to address the operational and reporting challenges faced by small and medium-sized clothing businesses. Many such businesses rely on spreadsheets, manual record-keeping, or disconnected tools for inventory management, customer tracking, supplier monitoring, and sales analysis. This creates inefficiencies, weakens reporting quality, and limits the ability of managers to make timely, data-driven decisions. GarmentIQ was developed as a centralized solution that combines transactional operations with analytical reporting.

The project was implemented as a full-stack system using ASP.NET Core for the web application, PostgreSQL for the relational database, Microsoft Azure for deployment, and Power BI for dashboard-based business intelligence. The system supports multiple user roles, including customer, staff, manager, and analyst, each with distinct workflows and permissions. Customers interact with product browsing, cart, login, and checkout functions, while internal users access role-based dashboards for viewing, updating, or analyzing operational data. Advanced SQL analysis was also completed across multiple levels, including basic reporting, trend analysis, window functions, common table expressions, business KPIs, and report-ready logic.

The final outcome is a functioning cloud-based system that integrates software development, database design, operational workflows, and analytics into a single project. The system demonstrates an end-to-end implementation of the software development life cycle and provides a practical foundation for future enhancement in reporting, automation, and decision support.

3.Table of Contents

Project Final Report 1
1. Cover Information 1
2. Abstract **1**

4. Introduction **2**
4.1 Project Background and Motivation 2
4.2 Problem Statement 3
4.3 Objectives and Scope 3
5. Related Work or Context **4**
6. Requirements Summary **4**
6.1 Stakeholders and User Needs 4
6.2 Functional Requirements Summary 5
6.3 Non-Functional Requirements Summary 5
7. System Design and Architecture **6**
7.1 High-Level Architecture 6
7.2 Component Descriptions 6
7.3 Key Design Decisions 7
8. Implementation **7**
8.1 Overview of Implementation 7
8.2 Software Implementation 8
8.3 Integration 8
8.4 Workflow Implementation 9
9. Testing and Evaluation **14**
9.1 Testing Strategy 14
9.2 Test Cases and Results 14
9.3 Defects and Fixes 15
9.4 Evaluation Against Requirements and Goals 16
10. Deployment and Maintenance **16**
10.1 Deployment Environment and Procedure 16
10.2 Backup and Recovery 16
10.3 Maintenance, Monitoring, and Updates 17
11. Project Management and Process Reflection **17**
11.1 Work Plan and Timeline Summary 17
11.2 Team Roles and Collaboration 17
11.3 Risk Management 18
11.4 Lessons Learned 18
12. Conclusions and Future Work **19**
12.1 Conclusions 19
12.2 Limitations 19
12.3 Future Work 19
13. References **19**
14. Appendices **20**
Appendix A – Power BI Dashboards 20
Appendix B – Project Links 20

---

4. Introduction

4.1 Project Background and Motivation

The apparel business environment depends on accurate coordination of product data, inventory availability, supplier relationships, customer activity, sales records, and operational reporting. In many small and medium-sized businesses, these activities are handled using separate spreadsheets, manual logs, or isolated applications that do not communicate effectively with one another. As a result,

inventory visibility is reduced, sales tracking becomes inconsistent, and reporting often takes place too late to support timely business decisions.

GarmentIQ was developed in response to this type of problem. The project was designed as a centralized system that combines a web-based operational platform with a structured relational database and a reporting layer. The intention was not only to manage transactions such as product records, sales activity, and customer interactions, but also to support analysis through SQL-based reporting and Power BI dashboards. This approach reflects a broader motivation to move from disconnected operational tools toward an integrated decision-support environment. The project also provided an opportunity to apply the software development life cycle in a realistic, business-oriented context, beginning with requirements gathering and ending with implementation, deployment, testing, and evaluation.

4.2 Problem Statement

Small and medium-sized apparel businesses often lack a centralized system to manage inventory, customer orders, supplier records, internal users, and sales performance. Because data is frequently stored in different places or maintained manually, it becomes difficult to track stock levels accurately, monitor operational activity, and generate reliable business insights. This reduces efficiency, increases the likelihood of data inconsistency, and weakens decision-making. The main problem addressed by this project was therefore the absence of a unified platform capable of supporting both operational management and data-driven analysis in an apparel business environment.

4.3 Objectives and Scope

The primary objective of GarmentIQ was to develop a centralized web-based system for apparel operations and sales intelligence. To achieve this, the project focused on several linked objectives: implementing role-based access for multiple user types, integrating a relational database to store operational data, supporting customer and internal workflows, enabling CRUD functionality where appropriate, deploying the system in a cloud environment, and preparing the system for analytical reporting using SQL and Power BI.

The agreed scope included a browser-based interface, authentication, role-based pages and workflows, a structured relational database, staff and manager operations, analyst read-only access, data handling for customers, products, categories, suppliers, sales, and orders, advanced SQL reporting, workflow documentation, and dashboard development. The project did not include a mobile application, production-grade payment gateway integration, advanced machine learning automation, or live third-party external integrations. These exclusions were kept outside the scope in order to maintain feasibility within the semester timeline and solo implementation model.

5. Related Work or Context

Retail and apparel systems generally fall into one of two categories. The first category includes transactional systems, such as e-commerce platforms or inventory management applications, that focus on product display, orders, and record maintenance. The second category includes reporting and business intelligence systems that focus on dashboards, summaries, and strategic analysis. In many practical settings, smaller businesses use separate tools for these purposes, which creates duplication of effort and weak traceability between operational activity and reporting outcomes.

GarmentIQ differs from a basic retail web application because it is not limited to browsing and order handling. It also includes role-based internal access, supplier and user-related operational records, structured database relationships, cloud deployment, workflow diagrams, and analytical preparation through advanced SQL and dashboards. At the same time, it differs from a dashboard-only analytics project because it is supported by an actual operational system that produces and manages the underlying data. This makes the project more complete than a simple CRUD application and more grounded than a standalone reporting artifact. It combines operational functionality with analytical capability in a single environment.

---

6. Requirements Summary

6.1 Stakeholders and User Needs

The system was designed around several key stakeholders, each with distinct operational or analytical needs.

STAKEHOLDER	ROLE IN SYSTEM	MAIN NEEDS
CUSTOMER	External user	Browse products, add to cart, log in or sign up, and complete checkout
STAFF	Internal operational user	Manage day-to-day records such as products, categories, order updates, and related operational actions
MANAGER	Internal administrative user	Monitor and update customers, suppliers, users, roles, sales, orders, and operational records
ANALYST	Internal reporting user	View system data for analysis, reporting, and insight generation without modifying records
DEVELOPER	System builder and maintainer	Design, implement, test, deploy, and improve the system

These stakeholder needs shaped the system architecture, permission model, workflows, and reporting design. Operational users required accuracy, usability, and controlled access, while analytical users required clean, query-ready data. Customers required a clear purchase flow, and the developer required a maintainable structure that could support both implementation and future extension.

6.2 Functional Requirements Summary

The main functional requirements of GarmentIQ are summarized below.

- **FR1:** The system shall allow authorized users to log in securely.
- **FR2:** The system shall support role-based access for customer, staff, manager, and analyst users.
- **FR3:** The system shall allow customers to browse products and view shop content.
- **FR4:** The system shall allow customers to add products to a cart and proceed to checkout.
- **FR5:** The system shall redirect unauthenticated customers to login or sign-up before final checkout.
- **FR6:** The system shall allow staff and managers to access dashboards after successful authentication.
- **FR7:** The system shall allow authorized internal users to create, read, update, or delete operational records according to their roles.
- **FR8:** The system shall store customer, product, inventory, category, supplier, user, order, and sales data in a relational database.
- **FR9:** The system shall support analytical reporting through SQL queries and dashboard-ready data structures.
- **FR10:** The system shall allow analyst users to review relevant business data without operational editing permissions.

These requirements were intentionally defined at both operational and reporting levels so that the project would address not only transactions, but also visibility and analysis.

6.3 Non-Functional Requirements Summary

The key non-functional requirements are summarized below.

- **NFR1 (Performance):** The system should respond quickly to normal user interactions and support practical operational use.
- **NFR2 (Security):** Internal access must require authentication, and role-based permissions must restrict actions by user type.
- **NFR3 (Usability):** The interface should be clear enough for customers and internal users to navigate without unnecessary complexity.
- **NFR4 (Reliability):** System data must be stored consistently and remain available across sessions.
- **NFR5 (Maintainability):** The system structure should support future changes in dashboards, workflows, or business logic.
- **NFR6 (Scalability):** The architecture should be capable of supporting future analytical features and extended data volume.
- **NFR7 (Deployment Readiness):** The application should function in a cloud-hosted environment rather than only in a local development environment.

These non-functional requirements influenced design decisions such as layered architecture, authentication handling, normalized database structure, modular logic, and cloud deployment.

7. System Design and Architecture

7.1 High-Level Architecture

GarmentIQ follows a layered full-stack architecture designed to separate user interaction, business logic, data persistence, and analytics. At the presentation layer, the system provides a web interface for customers and internal users. At the application layer, the backend processes requests, enforces workflows, validates access, and handles business rules. At the data layer, PostgreSQL stores structured business data across related entities such as products, categories, suppliers, customers, users, orders, and sales. Above this, an analytics layer supports SQL-based business analysis and dashboard reporting. Finally, the deployment layer places the application and database in Microsoft Azure to simulate a live cloud-based environment.

This architecture was selected because it balances practical implementation with maintainability. It allows operational functionality and analytical functionality to coexist while still keeping system responsibilities logically separated.

7.2 Component Descriptions

Presentation Layer
The presentation layer includes the visible interface used by customers and internal system users. It supports actions such as product browsing, cart interaction, authentication, dashboard access, and navigation across pages. This layer is responsible for collecting user input and sending requests to the backend for processing.

Application Layer
The application layer contains the core system logic. It manages request handling, authentication outcomes, page routing, workflow control, CRUD operations, and role-based behavior. This layer acts as the main coordinator between the frontend and database, ensuring that business rules are followed before data is read or written.

Data Layer
The data layer uses a PostgreSQL relational database. It stores structured entities required for the business domain, including products, customers, categories, suppliers, users, roles, sales, and orders. The relational model makes it possible to enforce connections between tables, maintain consistency, and support both transactional use and reporting use.

Analytics Layer
The analytics layer supports deeper business understanding. It includes advanced SQL queries for summaries, rankings, trends, segmentation, and reusable reporting logic, as well as Power BI

dashboards for visualization. This layer transforms raw records into meaningful operational and strategic insight.

Deployment Layer
The deployment layer includes Microsoft Azure App Service and Azure PostgreSQL Flexible Server. This layer enables the system to be accessed in a cloud environment and demonstrates the deployment considerations associated with a live application rather than only a local prototype.

7.3 Key Design Decisions

Several important design decisions shaped the final system.

The first major decision was to use a layered architecture. This improved separation of concerns and made the system easier to understand, develop, and extend. Rather than mixing user interface logic directly with storage logic, the system organizes responsibilities across presentation, application, data, and analytics layers.

The second decision was to use a relational database design. Because GarmentIQ needed to handle multiple connected entities such as users, customers, suppliers, products, categories, and orders, a relational approach was more suitable than an unstructured storage model. It also made analytical SQL possible at a higher level of depth.

The third decision was to implement role-based workflows. Customers, staffs, managers, and analysts do not interact with the system in the same way, and their permissions are not identical. Building separate workflows improved clarity and helped the system reflect realistic business roles.

The fourth decision was to include an analytics layer as part of the project scope. This moved GarmentIQ beyond a standard CRUD application and allowed it to support decision-making through advanced SQL and dashboards.

The fifth decision was to deploy the system in Azure. Although cloud deployment introduced extra complexity, it made the project more realistic and allowed evaluation of live behavior rather than only local functionality.

---

8. Implementation

8.1 Overview of Implementation

GarmentIQ was implemented through an iterative process that aligned with the planned project phases. The work began with requirements gathering and system analysis, followed by diagram creation, database preparation, and dataset development. The next stage focused on frontend structure and backend connectivity, after which authentication, dashboards, and real-time database interaction were added. Cloud deployment was then introduced, and the final stages emphasized workflow completion, advanced SQL analysis, and dashboard integration.

This implementation sequence aligned closely with the chosen architecture. The frontend was developed to provide the necessary user-facing pages and dashboards. The backend connected these pages to the database and enforced business rules. The database stored the underlying records, and the reporting layer was constructed on top of this operational structure. As a result, the implementation was not a collection of isolated parts, but an integrated progression from requirements to operational system to reporting-ready environment.

8.2 Software Implementation

The software implementation involved several major technologies and modules.

The web application was developed using ASP.NET Core, which provided the application structure, routing behavior, and server-side support needed for a role-based web platform. HTML, CSS, and limited JavaScript were used for interface presentation and page interaction. PostgreSQL served as the primary database because of its suitability for structured data, relationships, and reporting support. Database tools such as pgAdmin4 were used to manage schema configuration, connectivity, and data verification. GitHub was used for version control, while Microsoft Azure was used to host both the web application and the cloud database. Power BI was selected for dashboard and reporting output.

The software modules can be grouped into the following functional areas:

- Customer-facing shop and cart workflow
- Login and authentication handling
- Internal dashboards for staff, manager, and analyst roles
- Backend logic for controlled data operations
- Database integration for persistent business records
- SQL analytics for reporting and decision support
- Dashboard preparation for visual business intelligence

Each module contributes to a broader business purpose rather than acting as an isolated feature. For example, the customer shopping flow creates order-related data, which supports internal monitoring, which in turn feeds analytical reporting.

8.3 Integration

Integration was one of the most important aspects of the final system. The frontend interface, backend logic, relational database, workflows, SQL reporting layer, and cloud deployment all had to work together for the project to be meaningful. The integration process connected customer actions such as browsing and checkout with database storage and role-based internal access. It also connected operational records with reporting outputs so that the system could support both transactions and analytics.

From an end-to-end perspective, the system functions as follows. A user accesses the web interface and performs an action. The backend receives the request, applies validation and role logic, then reads or writes data to the PostgreSQL database. The updated records can then be reviewed through

internal dashboards or analyzed using SQL and Power BI. This integration supports a full operational-to-analytical flow rather than a partial prototype.

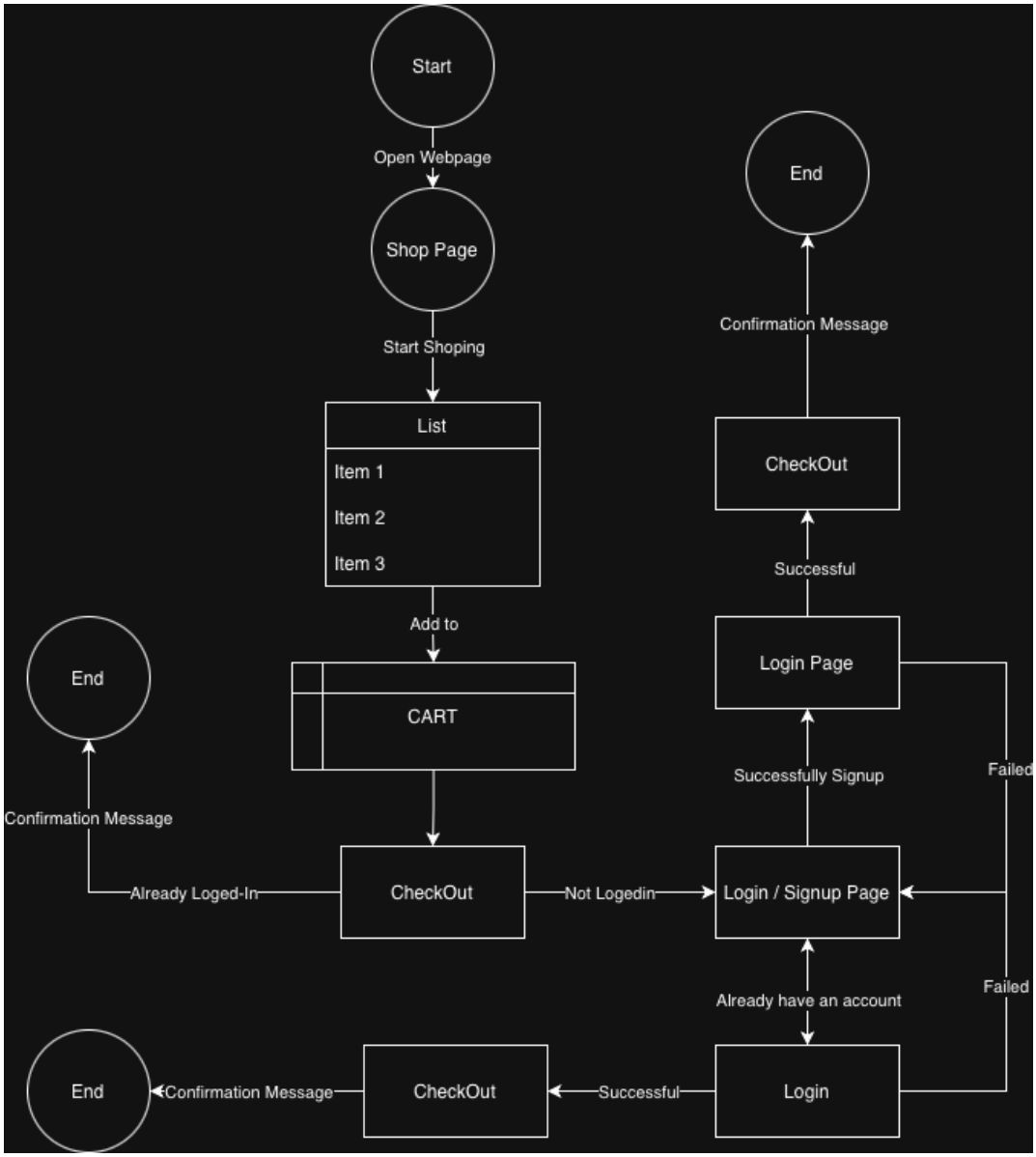
8.4 Workflow Implementation

To explain the behavior of the system clearly, four main workflow diagrams were created. These workflows document how each type of user interacts with the system, what decision points exist, what permissions are available, and how actions progress from start to end. Including these workflows also helps demonstrate end-to-end readiness, which was an area previously identified for improvement in earlier feedback.

8.4.1 Customer Workflow

The customer workflow begins when a user opens the website and enters the shop page. From there, the customer browses the available products and adds selected items to the cart. After reviewing the cart, the customer proceeds toward checkout. At this point, the system checks whether the customer is already authenticated. If the customer is logged in, the checkout process continues directly and the order is completed. If the customer is not logged in, the system redirects the user to a login or sign-up path.

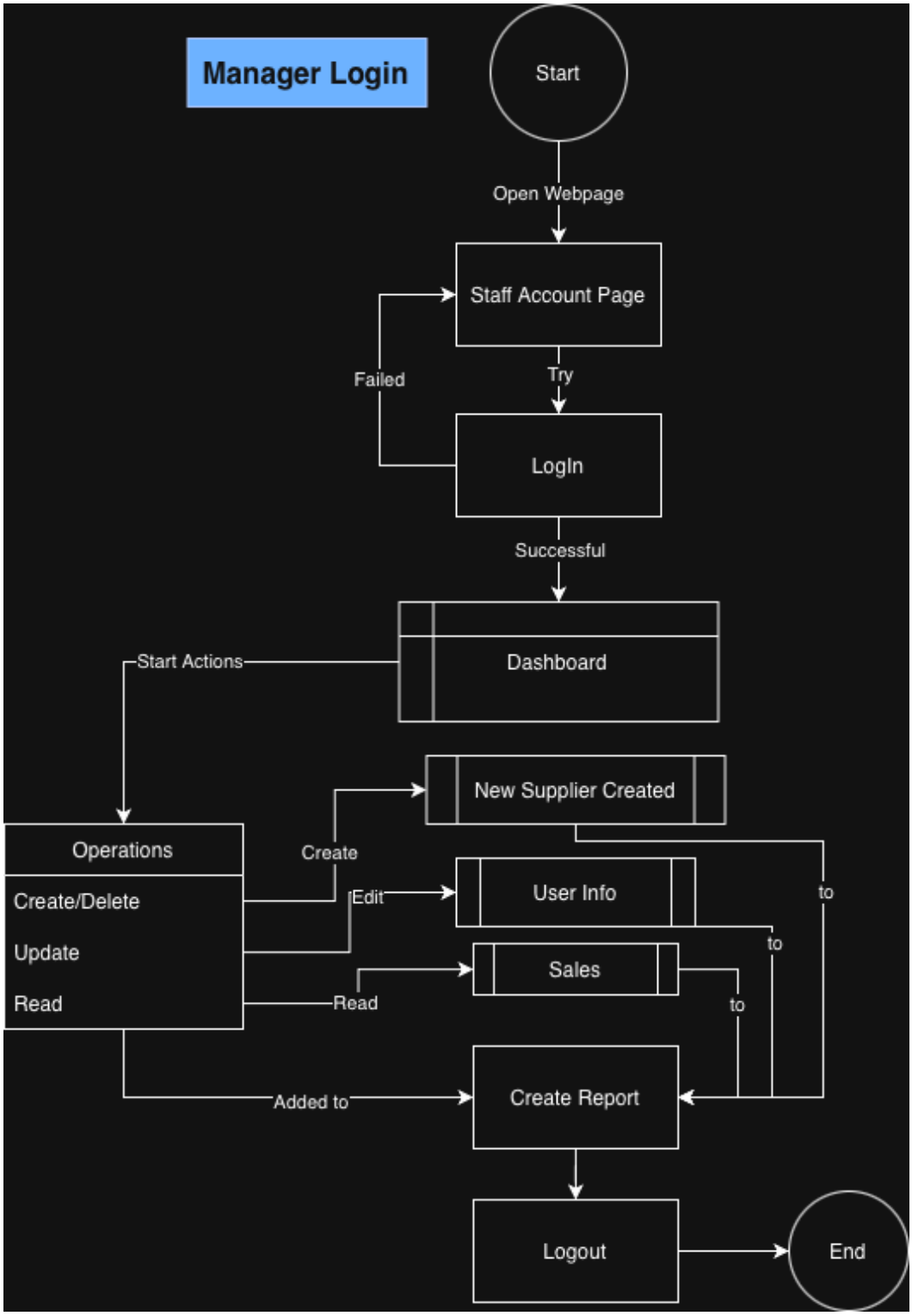
Within that path, the user either enters existing credentials or creates a new account. If credentials are entered correctly, the customer is returned to the checkout stage and can complete the order. If login fails, the workflow loops through the login or account-creation path until a valid account is established or a valid login occurs. The workflow ends when the checkout is completed successfully and a confirmation is produced.



8.4.2 Manager Workflow

The manager workflow begins with access to the manager login page. The manager enters credentials, and the system validates them. If the credentials are invalid, the workflow returns to the login step. This loop continues until valid credentials are provided. After successful authentication, the manager enters the manager dashboard.

Within the dashboard, the manager can review business records such as customer information, user data, supplier records, sales activity, orders, and other administrative information. Unlike the analyst role, the manager is not limited to observation. The manager can perform updates where permitted, such as modifying records, reviewing operational status, and managing administrative data. Once monitoring or updates are complete, the manager can log out, ending the workflow.

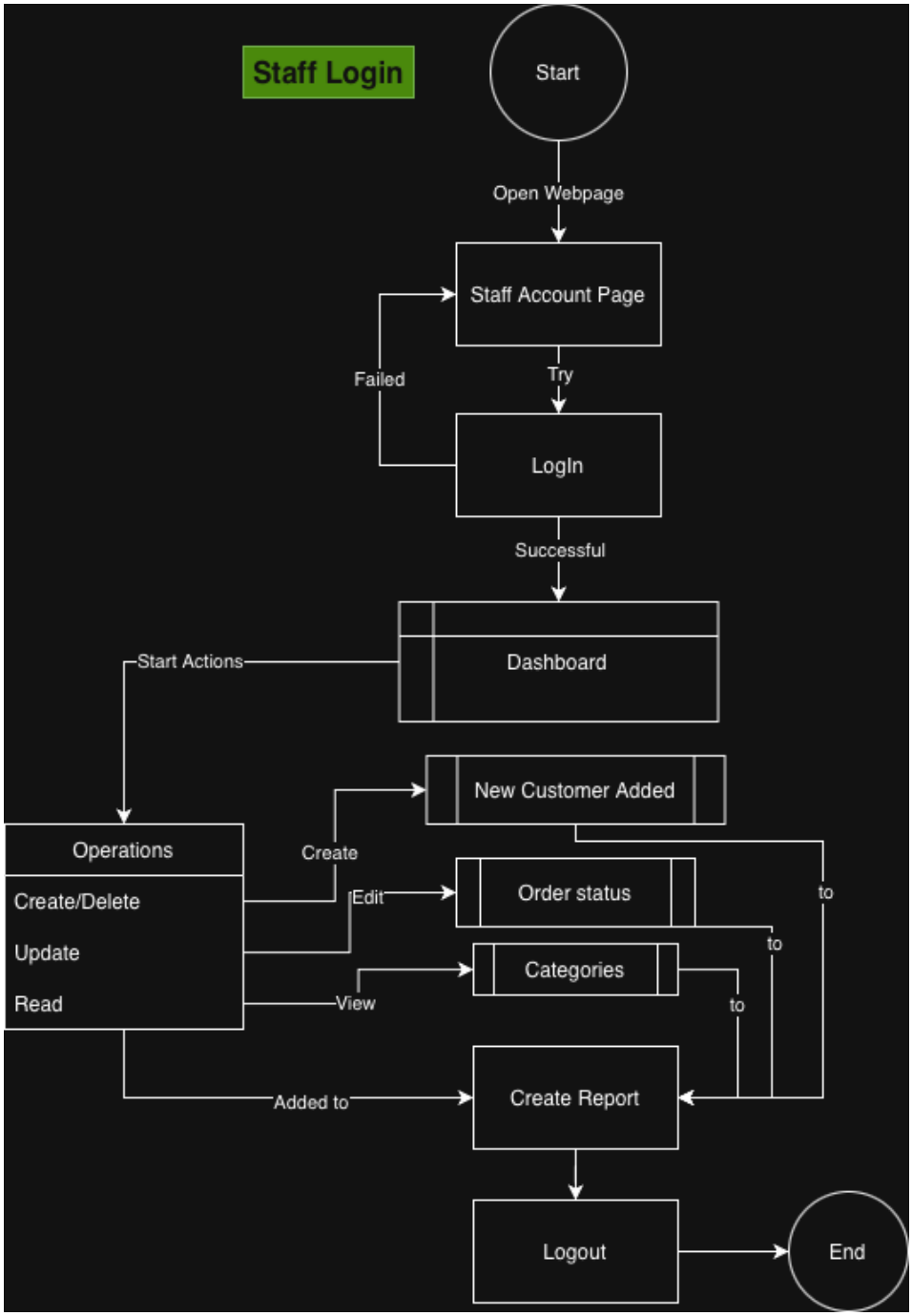


8.4.3 Staff Workflow

The staff workflow begins with staff access to the internal login page. As with the manager, invalid credentials cause the process to loop back until successful authentication occurs. Once logged in, the staff member enters the staff dashboard and begins operational interaction with the system.

The staff role is designed around day-to-day business activities. Depending on permissions, staff can perform actions such as reading records, updating data, creating or deleting certain operational

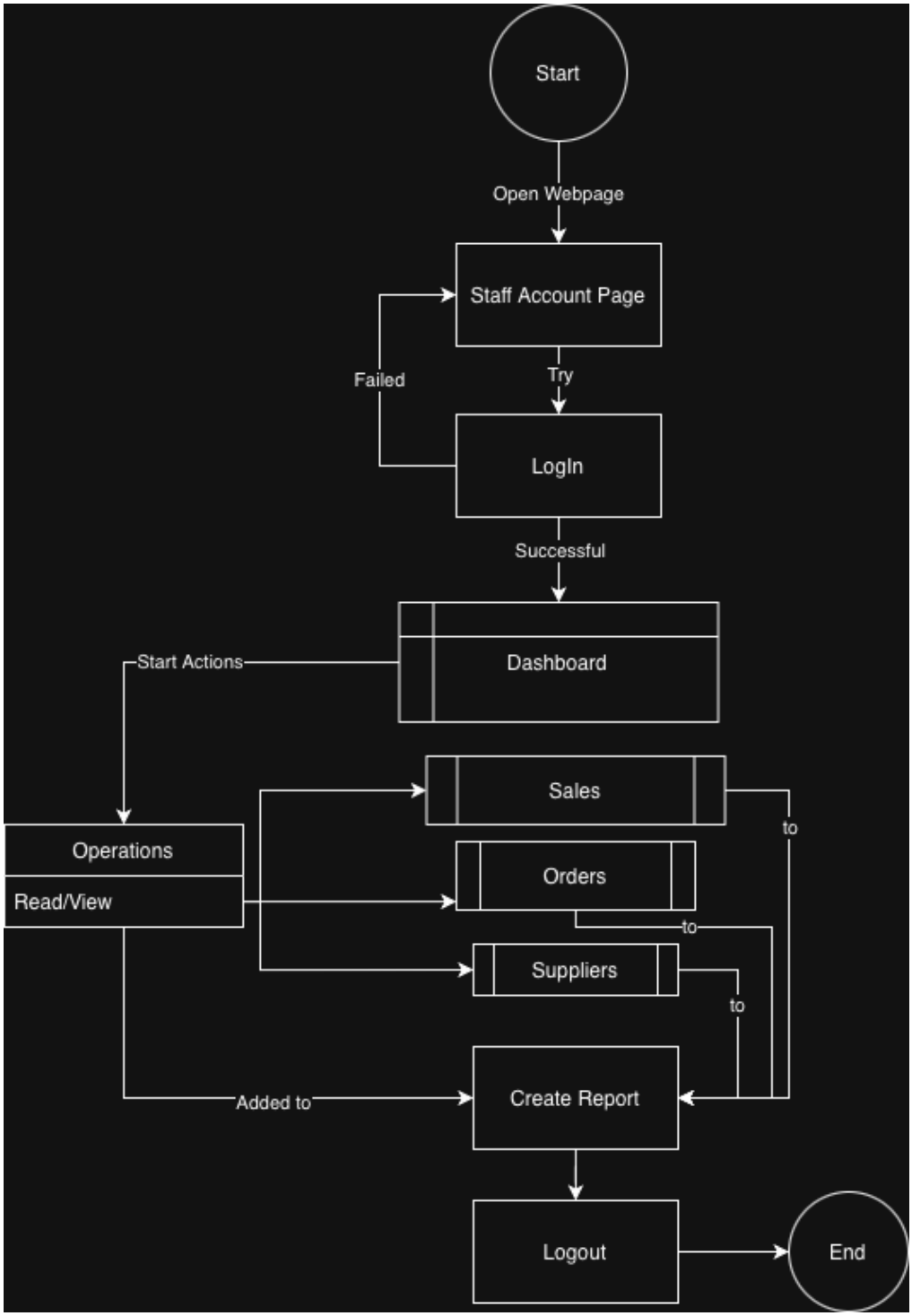
entries, managing categories or customer-related updates, and working with order status or other internal operational components. The workflow also supports report-related output where relevant. After operational actions are completed, the staff user logs out and the process ends.



8.4.4 Analyst Workflow

The analyst workflow is structured around information access rather than operational editing. The analyst begins by logging into the system. Failed authentication returns the analyst to the login step until valid credentials are entered. After successful login, the analyst enters the dashboard and views the available business data.

The analyst can review information such as sales performance, suppliers, orders, and other data used for reporting. However, unlike staff or manager users, the analyst role is primarily read-only and does not carry update or delete authority over operational records. This separation preserves system control while still allowing analytical access. After reviewing and using the relevant data for reporting purposes, the analyst logs out and the workflow ends.



9. Testing and Evaluation

9.1 Testing Strategy

The testing strategy focused on validating the system from both functional and integration perspectives. Because the project combined customer-facing features, internal workflows, database operations, cloud hosting, and reporting logic, testing needed to cover more than basic page loading. The overall approach emphasized manual scenario-based testing supported by direct observation of database behavior and query outputs.

Testing was organized into the following categories:

- Authentication testing
- Workflow testing
- CRUD and data consistency testing
- Database relationship testing
- Frontend-backend integration testing
- Cloud deployment testing
- SQL result validation
- Reporting readiness evaluation

This strategy was appropriate for the scope of the project because it allowed the major user journeys and core business functions to be tested end-to-end. It also aligned with the need to show more complete workflow evidence and final system readiness.

9.2 Test Cases and Results

The key test cases and results are summarized below.

Test Case	***Purpose***	***Expected Result***	***Actual Result***	***Status***
Customer opens shop page	Validate frontend product access	Products are visible and navigable	Shop page displayed correctly	Pass
Customer adds items to cart	Validate cart operation	Selected products appear in cart	Cart updated correctly	Pass
Logged-in customer checks out	Validate direct purchase flow	Checkout completes and order is stored	Completed successfully	Pass
Non-logged-in customer checks out	Validate redirect logic	User is redirected to login/sign-up	Redirect worked correctly	Pass
Invalid login attempt	Validate authentication control	User is returned to login page	System returned to login	Pass
Staff login and	Validate staff	Staff dashboard	Worked correctly	Pass

dashboard access	authentication and routing	opens after valid login		
Manager login and dashboard access	Validate manager authentication and routing	Manager dashboard opens after valid login	Worked correctly	Pass
Analyst login and read-only access	Validate limited reporting access	Analyst can view but not modify records	Worked correctly	Pass
Staff operational updates	Validate role-based data modification	Authorized updates are stored in database	Worked correctly	Pass
Manager administrative review/update	Validate manager privileges	Manager can view and update permitted records	Worked correctly	Pass
Database foreign key validation	Validate relational consistency	Related records remain consistent	Verified successfully	Pass
Azure deployment access	Validate live hosting availability	Application loads and connects in cloud	Mostly successful with minor inconsistencies	Partial Pass
SQL reporting outputs	Validate analytics correctness	Queries produce meaningful business outputs	Verified successfully	Pass

These results indicate that the core operational flows and reporting logic are functioning as intended, with the most notable remaining limitations concentrated in cloud behavior rather than core local functionality.

9.3 Defects and Fixes

Several defects and technical issues were identified during development and testing.

One recurring challenge involved authentication behavior and role-based routing. Because multiple user types needed different access paths, login validation and redirection had to be refined until the correct dashboard and restriction behavior was achieved. This was addressed through iterative backend testing and workflow review.

A second challenge involved database relationships. The project includes multiple linked entities, and early implementation revealed the need for continued refinement of structure and query behavior. This issue was addressed through ERD review, schema adjustment, and direct query testing.

A third challenge involved cloud deployment. While the application was successfully deployed to Azure, some behaviors did not always reflect as consistently in the live environment as in the local setup. This issue was mitigated through additional configuration and debugging, but remained a minor known limitation at the time of final evaluation.

The major known limitations are therefore not in the existence of the workflows themselves, but in the refinement of the deployed production behavior and the absence of automated test coverage.

9.4 Evaluation Against Requirements and Goals

The final system satisfies the major project goals and requirements at a strong level.

The objective of building a centralized apparel operations system was achieved through the creation of a multi-role web platform connected to a relational database. Role-based access was implemented through separate customer, staff, manager, and analyst flows. Operational data handling was achieved through dashboard interaction and controlled data access. SQL-based reporting was developed across six levels of complexity, demonstrating strong analytical capability. Power BI dashboard preparation added a visual business intelligence layer to the project. The application was also deployed to Azure, meeting the goal of cloud-hosted delivery rather than purely local execution.

The project therefore meets the core expectations set out in the original requirements and proposal documents while also responding to feedback from progress reports by strengthening workflow completeness, system integration, and testing explanation.

---

10. Deployment and Maintenance

10.1 Deployment Environment and Procedure

GarmentIQ was deployed using Microsoft Azure as the cloud hosting platform. The web application was published to Azure App Service, while the database was hosted using Azure PostgreSQL Flexible Server. This deployment model allowed the project to demonstrate how a full-stack application can operate in a live environment rather than only in a local development environment.

The deployment process included preparing the application for publishing, configuring the hosting environment, setting up the database connection, validating connectivity between the deployed application and the hosted database, and testing the live website for accessibility and workflow behavior. This procedure introduced additional technical complexity but also improved the realism and professional value of the final project.

10.2 Backup and Recovery

Backup and recovery were addressed at both the code and data-support level. Source code, development history, and project assets were maintained through GitHub version control. This allowed recovery of earlier code states and preserved progress across development stages. In addition, database-related resources such as schema files, SQL scripts, CSV exports, and project files served as supplementary data backups.

For a larger production environment, more formal backup automation would be required, including scheduled database backups, recovery testing, and cloud configuration snapshots. However, within

the scope of this project, the combination of version control and exported database-related resources provided an appropriate baseline recovery approach.

10.3 Maintenance, Monitoring, and Updates

The system can be maintained through a combination of code revision, deployment updates, and reporting extension. Application logic can be revised and redeployed through the GitHub-to-Azure workflow. Database maintenance can include schema refinement, data cleanup, indexing, and reporting optimization. Dashboards can be updated as the available data model expands.

Monitoring in a production sense would ideally include application logs, database health checks, error monitoring, and usage tracking. Although the current project does not implement a full production monitoring suite, its architecture supports future integration of such capabilities. The modular design also makes it practical to extend the system with additional workflows, more advanced analytics, or stronger security features over time.

---

11. Project Management and Process Reflection

11.1 Work Plan and Timeline Summary

The project followed an iterative, milestone-based structure that was aligned with course deliverables. Early stages focused on requirements, stakeholder analysis, and scope definition. The next stages emphasized system modeling, diagrams, and database preparation. Mid-stage work shifted to frontend and backend development, authentication, database integration, and dashboard planning. Later stages focused on cloud deployment, workflow completion, advanced SQL analysis, and final documentation.

Although the original plan remained broadly intact, some priorities evolved during implementation. For example, backend integration and authentication required sustained attention before some reporting or interface enhancements could be finalized. Azure deployment also introduced additional debugging effort. Despite these changes, the overall plan remained workable and continued to reflect the staged development process originally proposed.

11.2 Team Roles and Collaboration

The project was completed as a solo implementation. All major responsibilities, including requirements analysis, system design, database planning, workflow creation, frontend development, backend logic, testing, deployment, SQL reporting, dashboard preparation, and report writing, were carried out by Garv.

Because the project was completed individually, collaboration took a different form than in a multi-member team. Instead of distributing responsibilities across multiple people, the project required self-managed coordination across all technical and reporting tasks. Version control through GitHub,

regular progress reporting, and structured milestone planning served as the main internal management mechanisms.

11.3 Risk Management

Several risks were identified during planning and development. One major risk was database relationship complexity, since an incorrectly designed schema would affect both operational accuracy and analytical reporting. This was managed through early diagram creation, review of relationships, and continued query testing. Another risk was the technical difficulty of authentication and role-based behavior, which could delay dashboard access and workflow clarity. This was managed through iterative implementation and validation. A third risk was cloud deployment instability, particularly differences between local and live behavior. This was managed through staged deployment and debugging.

The main time-related risk was the fact that the project was implemented by one person. This made prioritization essential. Core functionality, integration, and system readiness were treated as mandatory, while highly advanced extras were kept outside the defined scope unless feasible.

11.4 Lessons Learned

This project provided several important lessons across technical execution, project planning, and system thinking.

A major technical lesson was the importance of strong database design before implementation. Once frontend and backend integration begins, weak relationships or unclear schema design can create problems across the entire system. Early diagram work therefore proved valuable.

A second lesson was that cloud deployment adds a separate layer of complexity beyond coding. A system that works locally is not automatically ready for a hosted environment. Configuration, connection handling, and live testing all require attention.

A third lesson was the value of combining operations and analytics in one project. Building only a frontend system would have limited the business depth of the work, while building only dashboards would have limited the realism of the data source. Integrating both made the project stronger and more representative of real business systems.

Finally, from a project management perspective, the work reinforced the value of staged milestones, iterative testing, and maintaining documentation throughout development rather than only at the end.

12. Conclusions and Future Work

12.1 Conclusions

GarmentIQ successfully evolved into a functioning web-based apparel operations and sales intelligence system. The final project integrates customer interaction, internal role-based workflows, relational data management, cloud deployment, advanced SQL reporting, and dashboard preparation in a single system. It demonstrates a full software development life cycle from requirements gathering to final implementation and evaluation. The project also shows how operational business systems can be strengthened by analytical capability rather than limited to record management alone.

12.2 Limitations

Despite its strengths, the current system has several limitations. Some cloud deployment behavior requires further refinement to achieve the same consistency observed in local testing. Automated testing was not implemented, so evaluation relied mainly on structured manual testing. The dashboard layer depends on the final insertion and presentation of Power BI visuals. The system also does not include production-grade payment integration, mobile optimization, or AI-based automation, all of which were intentionally excluded from the current scope.

12.3 Future Work

Future improvements could extend GarmentIQ in several directions. One improvement would be the addition of real-time low-stock alerts or automated notifications. Another would be the implementation of formal automated testing to support stronger quality assurance. The reporting layer could also be expanded with richer dashboard interaction, more KPI definitions, and drill-down analysis. Additional work could improve mobile responsiveness, add customer account history, strengthen internal audit logging, and introduce workflow automation for repetitive operational tasks. Over time, predictive or AI-based features could also be layered onto the existing operational and analytical foundation.

---

13. References

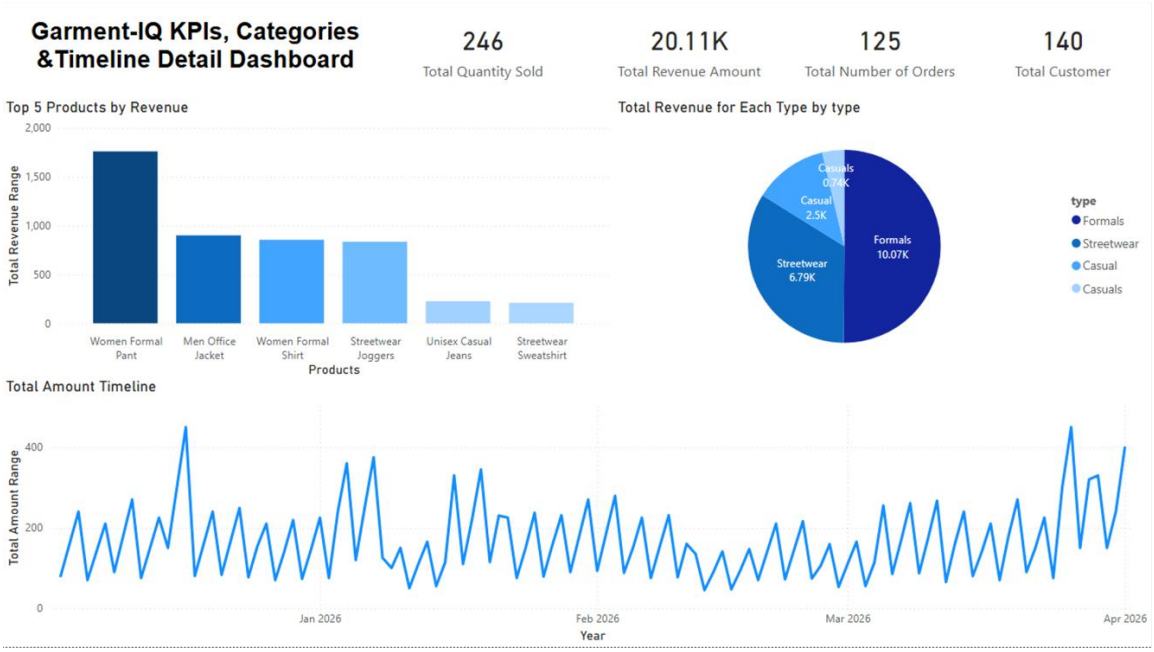
Requirements Gathering Report.
Project Proposal Report.
Weekly Progress Report 1.
Weekly Progress Report 2.
Weekly Progress Report 3.
Weekly Progress Report 4.
Final Project Report Template.
Final Project Report Rubric.
MySQL-GarmenIQ-DataAnalysis.sql.

---

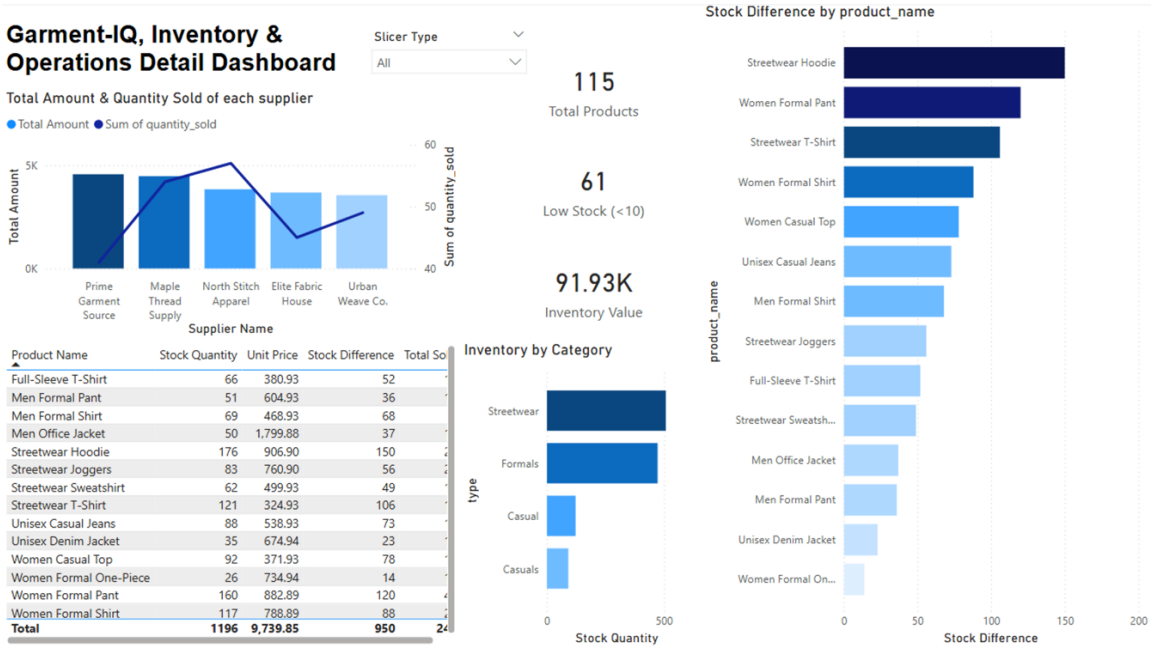
14. Appendices

Appendix A – Power BI Dashboards

- Dashboard 1



- Dashboard 2



Appendix B – Project Links

GitHub Repository: ClickHere

Live Website: ClickHere